ALLART Noah
ARMBRUSTER Loup
KHODJAOUI Elias

IUT Nancy-Charlemagne
3ème année

Groupe : RA-IL-1

https://github.com/Kuuro57/VR-4D_Projet_Tutore

Projet Tutoré :
S9 : Création d'une application en VR
Rendu final

Tuteur du projet : DUPONT Laurent (laurent.dupont@loria.fr)
Responsable des projets : BINET Julien (julien.binet@univ-lorraine.fr)

Table des matières

Table des matières 2
Introduction 3
Cahier des charges et objectifs 4
Analyse initiale 5
Rétrospectives sur les itérations 7
Itération 1 : Gestion des formes 7
Itération 2 : De la 3D à la 4D 10
Journée portes ouvertes 12
Itération 3 : De la 4D à la VR 13
Itération 4 : Ergonomie en 4D 15
Itération 5 : Dernières fonctionnalités 15
Itération 6 : Présentation aux BUT 1 16
Avis du groupe sur le développement 17
ALLART Noah : 17
ARMBRUSTER Loup : 17
KHODJAOUI Elias : 17
Perspectives : fin du projet 18
Conclusion finale 19
Annexes 19

Introduction

Ce document a pour but de récapituler l'intégralité de notre travail sur notre projet tutoré. Nous partirons du cahier des charges, tout en retraçant l'avancée du projet itération par itération.

Le sujet de notre projet est la réalisation d'une application en VR (réalité virtuelle) capable de rendre possible la visualisation d'une forme géométrique quelconque en 4D dans un espace en 3D. Concrètement, l'objectif de ce projet est de rendre la visualisation et la compréhension d'un objet en 4D plus intuitive.

Pour ce faire, nous avons à disposition un casque VR (Meta Quest 3) pour visualiser les objets géométriques dans un espace 3D. Pour le développement de l'application en elle-même, nous utilisons le langage JavaScript, et plus précisément la libraire Babylon.js. Cette bibliothèque est un moteur de rendu 3D pour les navigateurs web, et qui permet aussi nativement de créer et modéliser un environnement 3D compatible avec la VR.

Ce sujet repose sur deux piliers : la maîtrise technique de Babylon.js pour la réalité virtuelle et l'application de concepts mathématiques, tels que les matrices de rotation et les calculs de vecteurs.

Cahier des charges et objectifs

Pour mener à bien la réalisation de ce projet, nous avons eu quelques contraintes données par notre tuteur de projet.
Comme dit précédemment, nous devons utiliser la libraire Babylon.js pour avoir un moteur de rendu 3D déjà utilisable, sans quoi nous aurions dû le refaire de toute pièce, ce qui n'est pas chose facile compte tenu du temps donné pour réaliser l'application. Une deuxième contrainte est l'utilisation du casque VR pour visualiser la forme 4D, ce qui nous oblige à nous renseigner davantage sur les fonctionnalités de Babylon.js pour faire tourner notre application sur un casque VR.
L'objectif minimal à atteindre pour la fin du projet est la visualisation d'une forme 4D au travers du casque VR, ainsi que la possibilité d'interagir avec la forme 4D dans l'espace 3D (rotations, déplacement, agrandissement et rétrécissement).

Pour avancer efficacement, nous avons divisé le projet en plusieurs objectifs que nous avons catégorisé :

Objectifs critiques	**Objectifs complémentaires**
Projection d'une forme 4D (en 3D) dans la scène Babylon.js	Modéliser une scène Babylon.js en 3D
Rotation / Translation d'une forme 4D	Rotation d'un objet 3D à l'aide de matrices de rotations
Affichage d'une interface des différents plans de la figure 4D	Translation d'un objet 3D
Affichage d'une forme 4D en VR	Gestion des déplacements en VR (manettes / caméra)
	Affichage d'une forme 3D en VR

Ces objectifs sont là pour avoir un fil conducteur que nous pouvons suivre, et qui nous permettent aussi de nous faire une idée du bon avancement ou non du projet par rapport au délai fixé.

Analyse initiale

Avant de démarrer le projet, nous avons dû préparer le terrain en faisant une analyse initiale des contraintes, objectifs, étudier l'existant et explorer les défis techniques qui nous attendaient (surtout pour la partie mathématique).
Dans un premier temps, nous avons lu la documentation de Babylon.js pour essayer de comprendre son fonctionnement, puis nous avons développé des environnement 3D simples pour nous familiariser avec le moteur de rendu 3D. En parallèle, nous avons fait des recherches sur internet pour voir ce qui avait été fait avec Babylon.js, et plus précisément ce qui existait par rapport à la 4D. Nous n'avons trouvé qu'une seule vidéo YouTube montrant un environnement 3D au travers d'un casque VR affichant une forme en 4D. La seule différence avec notre projet était que l'application tournait grâce à Unity (un moteur de rendu 3D généralement utilisé pour le jeu vidéo) et non pas avec Babylon.js.

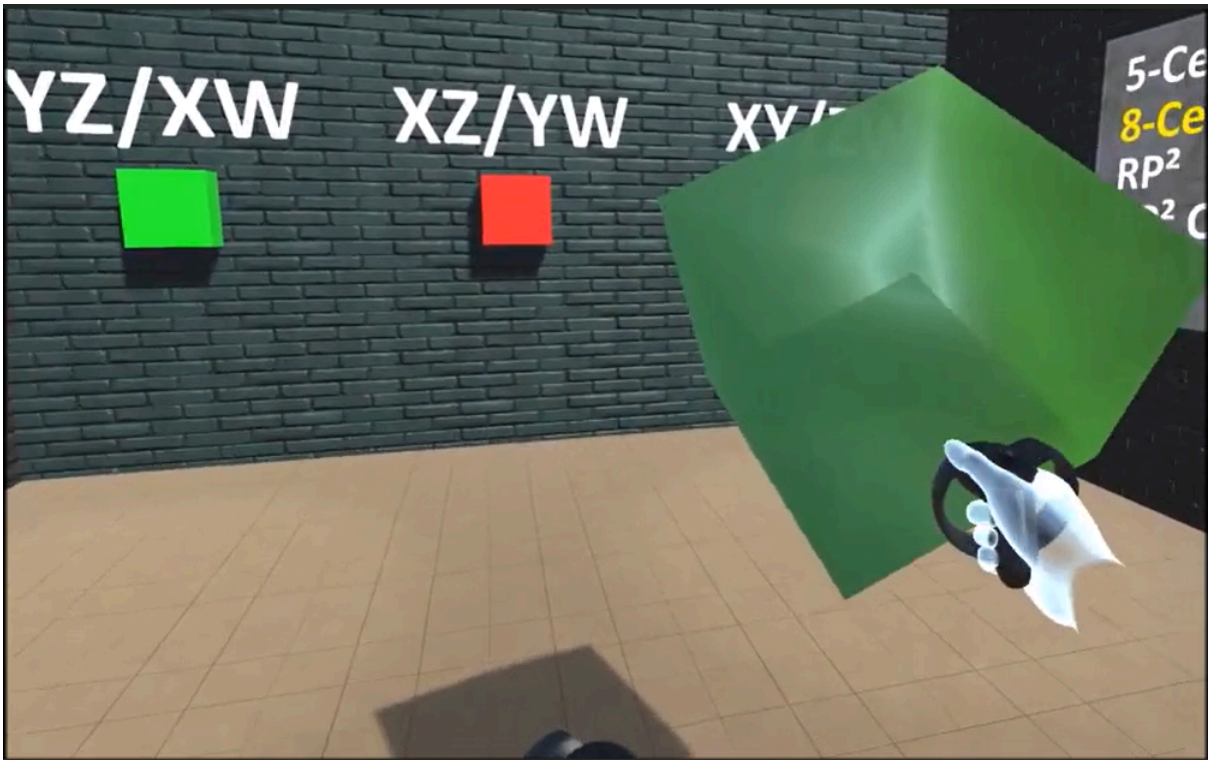

Figure n°1 - Visualisation d'une forme 4D sous Unity

Dans un second temps, nous nous sommes penchés sur l'étude technique sur l'existence de solutions. Cette partie nous a été d'une grande aide pour la suite, car c'est grâce à ces recherches que nous avons pu avancer rapidement sur le développement pure de l'application. Nous avons recherché sur beaucoup de sites comment fonctionnait la quatrième dimension, mais aussi comment était-il possible de visualiser une forme en 4D dans un espace 3D. Nous avons finalement trouvé le concept de plan projectif, qui permet de passer d'une dimension N à une dimension N-1 (exactement ce qu'il nous faut pour passer de la 4D à la 3D).

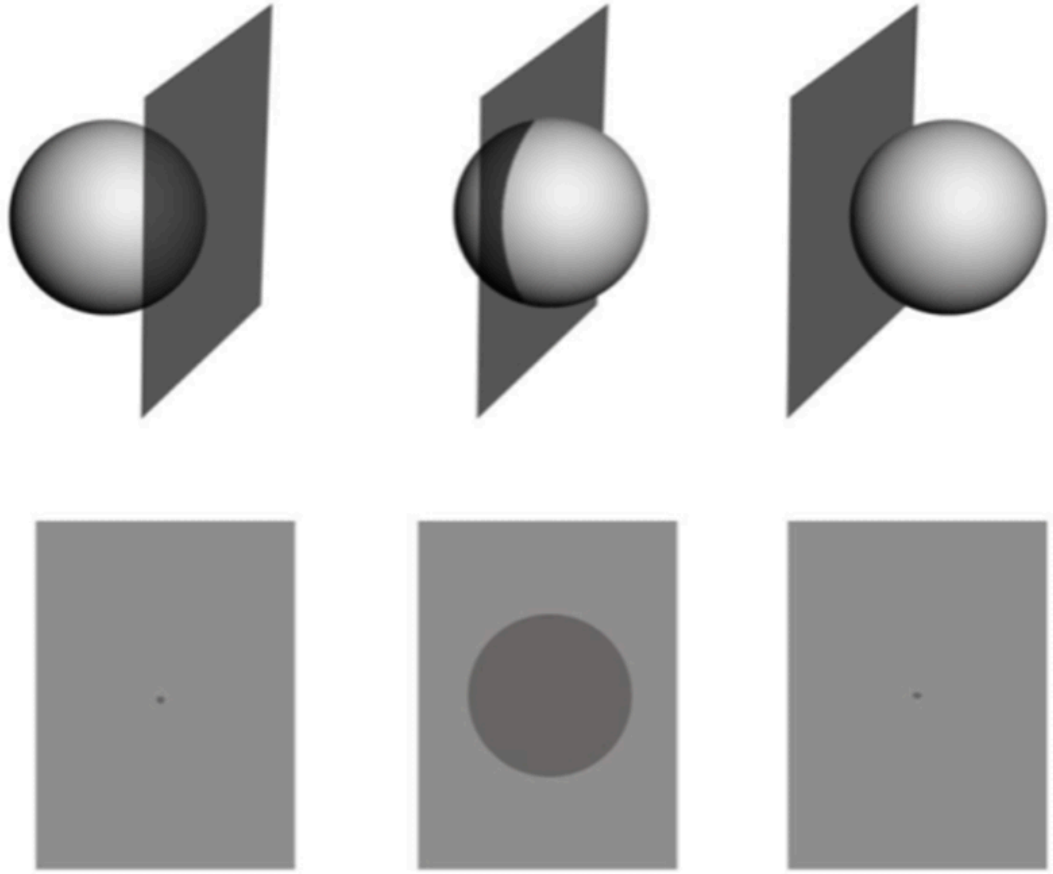

Figure n°2 - Exemple de plan projectif (3D → 2D)

Une fois celà fait, il fallait que l'on puisse effectuer des opérations sur notre forme 4D. Après de plus amples recherches, nous avons trouvé les matrices de rotations pour faire pivoter une forme 4D selon un plan. Pour ce qui est de la translation d'une forme 4D, c'est la même chose qu'en 3D.

Pour finir, nous avons étudié plus en profondeur la façon dont fonctionne Babylon.js pour représenter des objets dans son espace 3D. En effet, Babylon.js utilise des Vector3 (des vecteurs avec 3 coordonnées) pour positionner un élément dans l'environnement. Or nos formes en 4D ont 4 coordonnées (x, y, z, w). Pour représenter nos objets en 4D dans Babylon.js, nous avons imaginé créer des objets représentant nos formes 4D que nous passions en 3D (en utilisant des Vector3) avec l'utilisation d'un plan projectif, pour pouvoir afficher notre forme dans l'espace 3D.

Rétrospectives sur les itérations

Itération 1 : Gestion des formes

Pour la première itération, avec nos connaissances en javascript et notre analyse de la librairie Babylon.js, nous avons commencé par instancier des formes (un cube pour commencer).

Après avoir compris les bases, nous avons décidé de définir des classes pour plusieurs raisons :

- Générer facilement une forme sans avoir à toucher directement à la 3D (instancier les points est suffisant)
- Pour un travail collaboratif

En effet, un système de classes nous a permis de diviser une fonctionnalité (cube : développé par Elias) en plusieurs sous-classes pour définir une figure géométrique:

- Sommets (développé par Loup)
- Arêtes (développé par Noah)
- Faces (développé par Loup)

Une fois un cube affiché facilement en quelque lignes de codes (à l'aide d'une méthode de génération), notre but suivant a été d'interagir avec la forme à l'aide d'une interface :

- Translation (développée par Loup)
- Homothétie (développée par Elias)
- Rotation 3D (développée par Noah)

De plus, nous souhaitions ajouter différentes méthodes pour faciliter la suite du projet :

- Nous avons changé l'architecture, passant d'une classe Cube a une classe plus générique Forme capable d'afficher une forme en 3D comme en 4D (développée par Noah)

- Nous avons codé une méthode pour générer une figure générique en donnant la liste de ses sommets et arêtes (donnés dans un fichier JSON) (développée par Elias)

- Nous avons affiché les différentes projections 2D du cube dans les coins de l'écran (développée par Loup)

- Nous avons séparé la création de figures en différentes méthodes de la forme "load…From…()" pour générer chaque figure. Comme chaque figure a tous ses paramètres en commun, son affichage et les transformations qui lui sont appliquées restent les mêmes peu importe la figure (à l'exception de la rotation 4D) (développée par Loup, Noah et Elias)

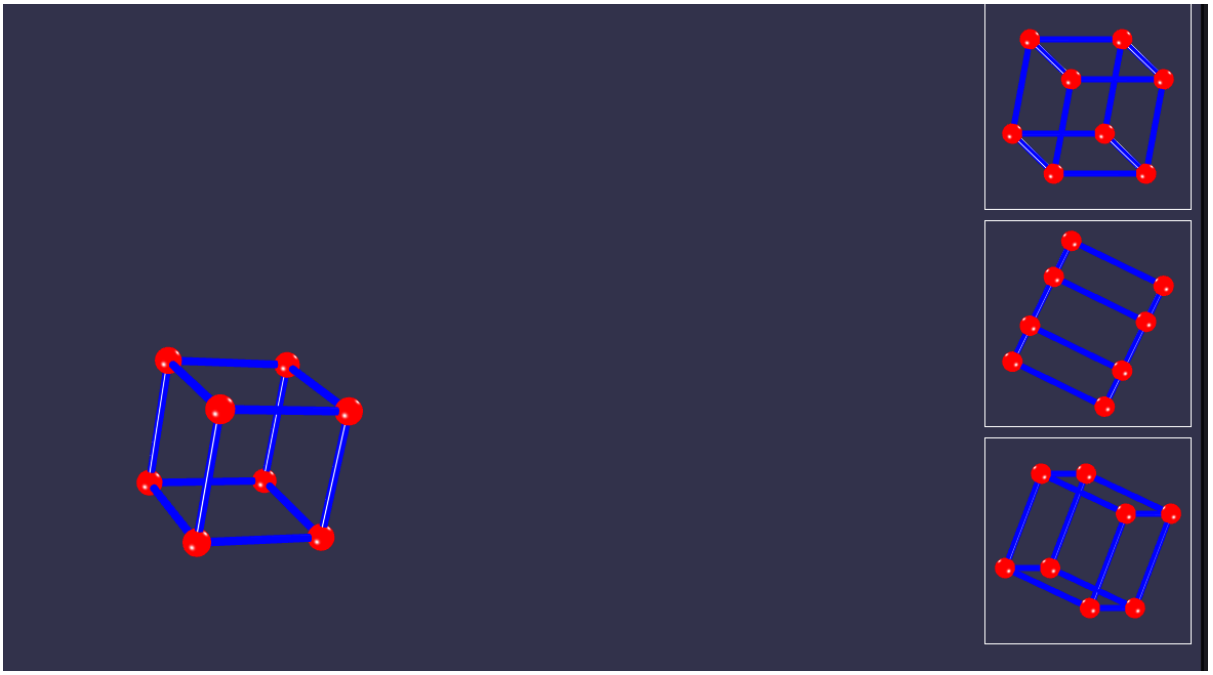

Figure n°3 - Affichage d’un cube et de ses projections 2D (vue fil de fer)

Cependant, avec le recul de 3 itérations, nous nous sommes rendus compte que l’utilisation de fichiers JSON est obsolète, et nous hésitons à l’améliorer ou à l’abandonner complètement.

Cette première itération a été en grande partie de la découverte, mais nous avons très bien distribué le travail, nous permettant de compléter toutes ces fonctionnalités pour la première itération. Nous avons donc pu présenter une scène BabylonJS avec un cube où toutes les transformations étaient codées correctement.
De plus, nous étions capables à la fin de l’itération d’afficher un hypercube (4D), de le déplacer, mais pas de le faire tourner.

Itération 2 : De la 3D à la 4D

Lors de la deuxième itération, comme toutes les opérations primaires étaient codées, nous avons pu nous concentrer uniquement sur la réflexion et le développement en 4D. Nous avons beaucoup discuté de comment implémenter les différentes figures, de comment elles se représentent et comment afficher leurs projections sur l'écran. En effet, la grande différence avec les formes 3D est que l'on ne peut pas afficher la figure en elle-même, mais seulement afficher une projection 3D d'une de ces formes 4D.

Après nous être entretenus avec notre tuteur Laurent Dupont, nous avons choisi "d'aplatir" une des 4 dimensions pour afficher les 3 dimensions restantes. En effet, en fonction de la projection, nous n'affichons pas les 3 mêmes dimensions. rendant chaque projection différente.

En termes d'organisation, cette itération fut plus compliquée, étant donné que l'on modifiait les mêmes fichiers pour les mêmes fonctionnalités, nous forçant à résoudre plusieurs merge git, nous ralentissant.

Cependant, chaque membre cherchait à faire quelque chose d'utile :

- Ajout de faces à l'hypercube et affichage des projections 3D (développé par Loup)
- Réglage de bugs et déplacement de l'hypercube (développé par Noah)
- Ajout de la rotation 4D et codage d'une sphère et hypersphère (développé par Elias)

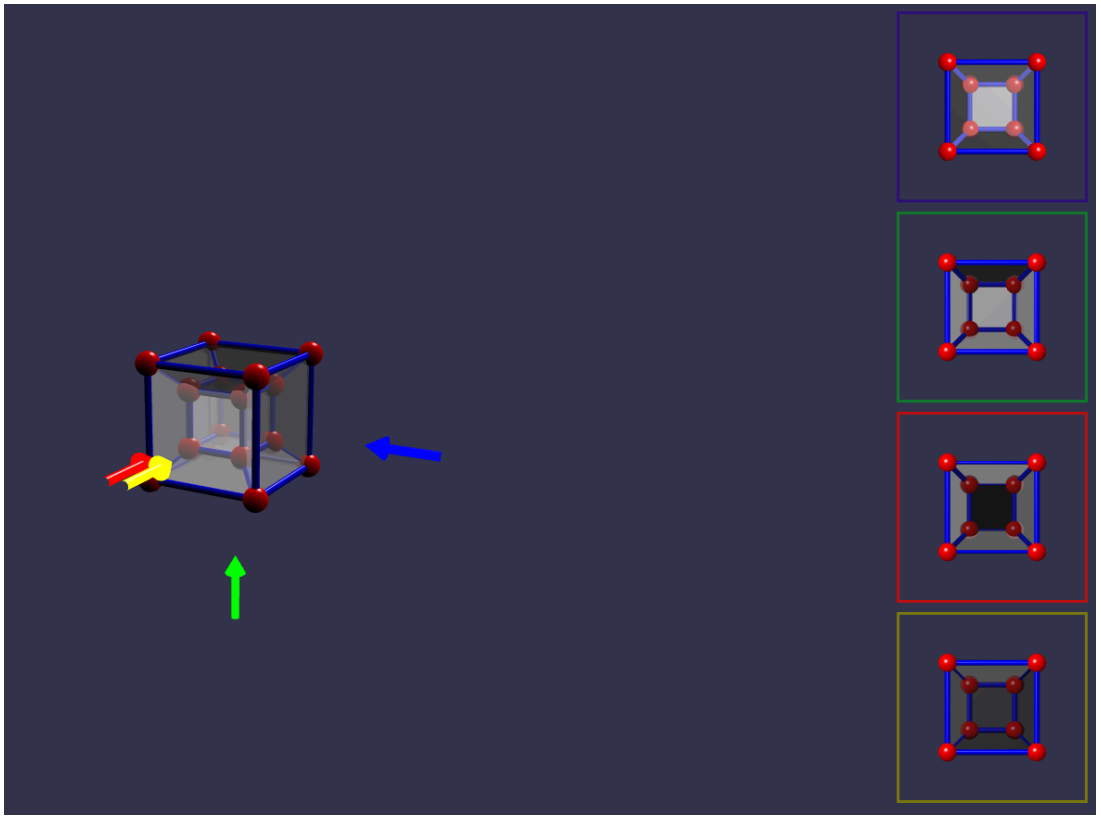

Figure n°4 - Affichage d’un hypercube et de ses différentes projections 3D

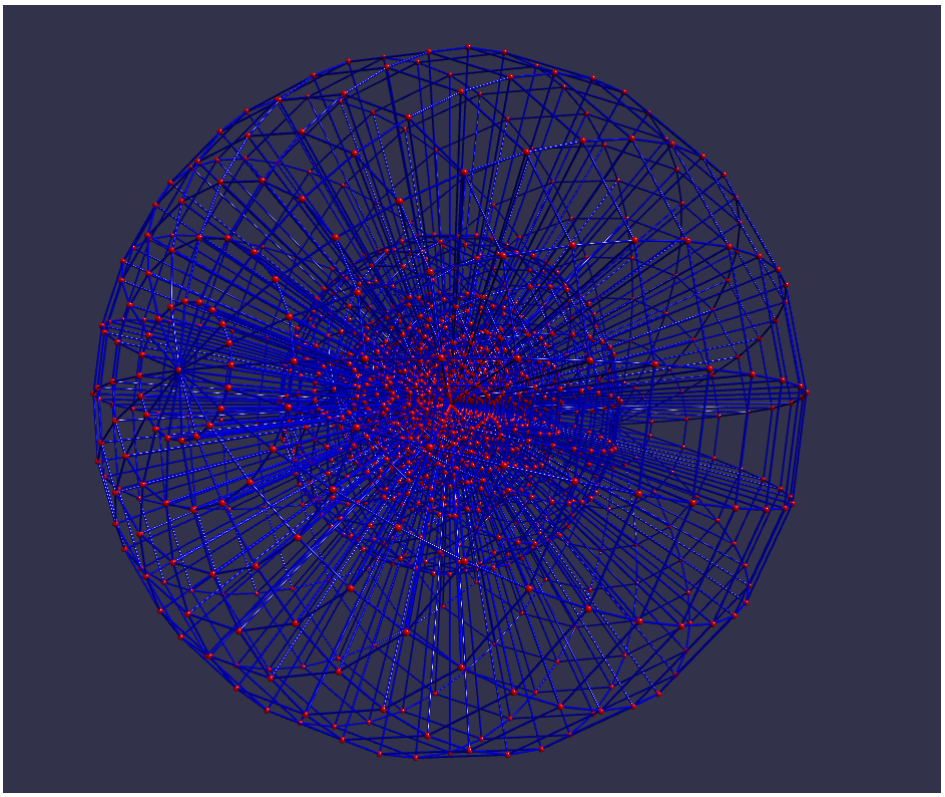

Figure n°5 - Affichage d’une hypersphère de près de 8000 points (20 sommets par segment)

Dans cette itération, nous nous sommes rendus compte que notre interface, bien que fonctionnelle, était peu utile pour le passage imminent à la VR. De plus, les contrôles s'étendaient sur une centaine de lignes de code, nous poussant à dupliquer les pages internet pour la 3D et la 4D (hypercube/hypersphère), plutôt que de réécrire tout notre code.

Il nous est apparu qu'il était nécessaire d'ici l'itération suivante d'avoir une page d'affichage propre, avec des contrôles plus simples (touches du clavier) facilitant le passage à la VR.

Nous sommes parvenus, malgré ces problèmes, à délivrer pour la fin de l'itération la 4D fonctionnelle (à l'exception de bugs liés aux rotations et translations en 4D), avec comme objectif suivant de commencer à passer en VR, tout en perfectionnant la 4D.

Journée portes ouvertes

Entre l'itération 2 et 3, nous avons eu le plaisir de participer à la journée portes ouvertes de l'IUT Nancy Charlemagne.

Dans ce cadre, nous avons pu présenter notre projet à des visiteurs et d'autres étudiants de l'IUT, recueillant par la même occasion des retours utilisateurs pour orienter la suite de notre projet.

En effet, nous avons remarqué notre besoin constant de recharger la page pour réinitialiser l'état de la figure. Une piste d'amélioration serait donc d'ajouter un bouton reset sur la page.

D'autre part, notre tuteur nous a proposé de colorer les faces différemment pour aider les utilisateurs à comprendre ce qui se passe lors des rotations.

Itération 3 : De la 4D à la VR

Pour cette troisième itération nous avons pu passer à une autre grosse partie de notre projet qui est le passage de la notre application web à une application en réalité virtuelle.

Pour appréhender le passage à la VR nous avons commencé par un refactor de notre code et par un retrait du menu d'interaction (fait par Noah) qu'il y avait auparavant pour relier chaque interaction à une touche du clavier puisqu'un des objectifs du passage à la VR est de pouvoir utiliser nos mains pour interagir avec la forme 4D.

En ce qui concerne la VR, nous avons commencé par chercher du code dans la documentation pour voir comment l'implémenter dans notre code. Nous avons été d'ajouter au projet un nouvel index auquel le casque VR peut se connecter (uniquement en https avec besoin de génération de certificat), lancer une "expérience VR" (mode déplacement VR) où l'on affiche un cube, soit une forme que l'on instancie nous même. (développé par Elias, Noah et Loup)
Nous avons commencé à relier les boutons des manettes à des contrôles temporairement, mais ils ne sont pas parfaits et présentent encore quelques problèmes notamment sur les rotations. De plus, nous n'avons pas encore créé des contrôles pour la VR sans manettes.

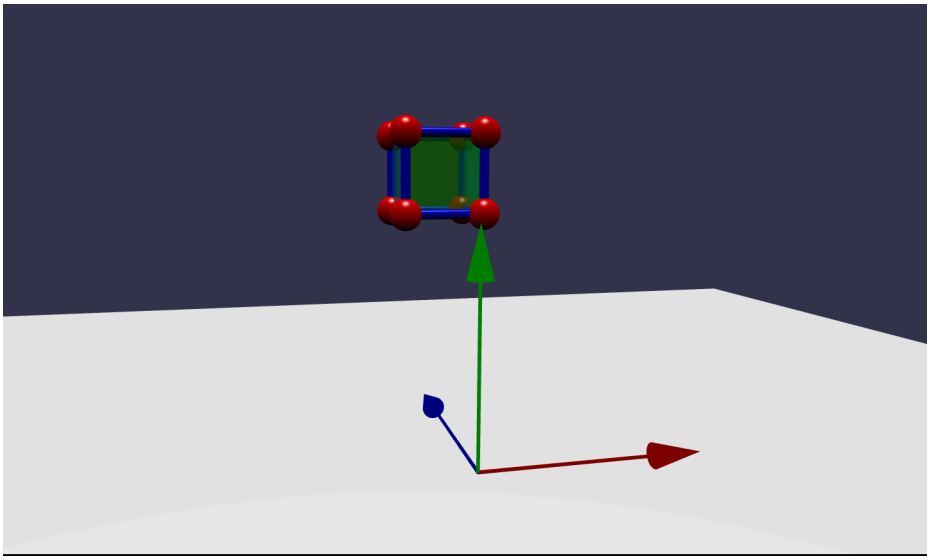

Figure n°6 - Vue VR d'une scène exemple, avec un cube pouvant être mû

Parallèlement, nous avons bien ajouté une coloration des faces (développée par Loup) comme recommandé par notre tuteur pour avoir une meilleure visibilité et compréhension de ce qu'il se passe lorsque l'utilisateur interagit avec la forme 4D.
Nous avons aussi ajouté un pentachore (ajouté par Elias) qui est un tétraèdre (pyramide) en 4D, qui est une des formes géométriques qui contient le moins de faces, d'arêtes et de sommets, considérée comme la figure 4D la plus "simple".

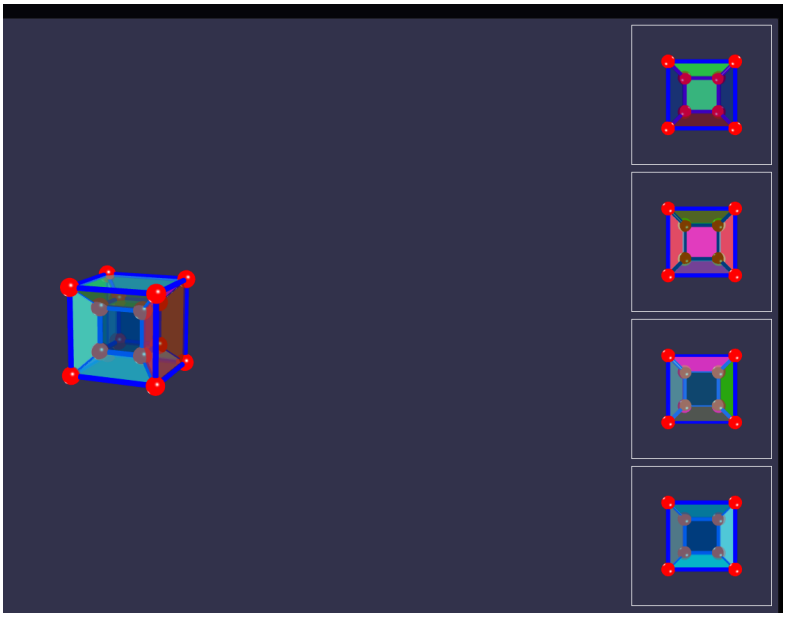

Figure n°7 - Hypercube final avec faces colorées

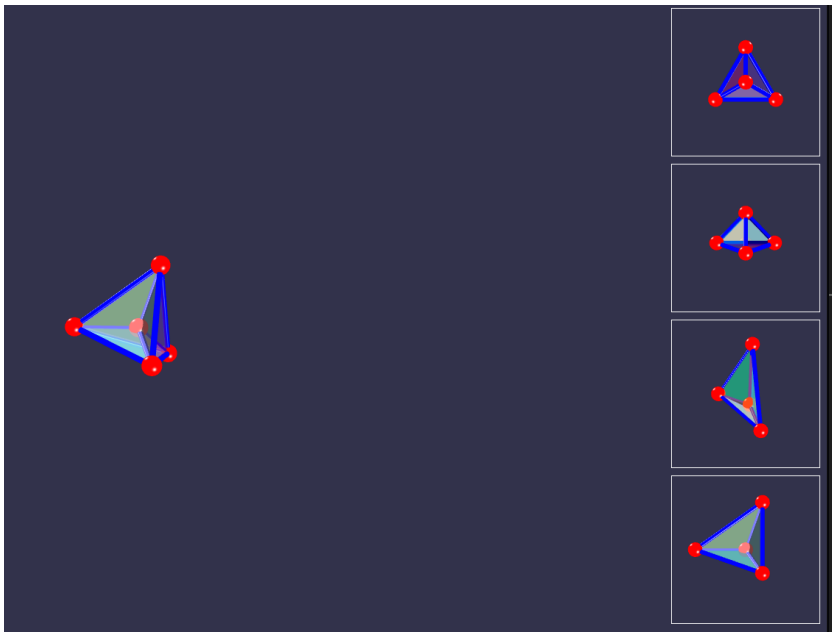

Figure n°8 - Affichage d'un pentachore

Itération 4 : Ergonomie en 4D

L'itération 4 était une semaine plus courte que les autres. En effet, notre soutenance d'itération 3 était au début de la semaine 4, et nous n'avons eu que 2 jours de développement. Ainsi l'avancée de cette itération a été plus mineure, et était concentrée sur des ajouts simples et de la documentation du code :

Notre programme a pu commencer à gérer les fichiers PLY (ajoutés par Elias). L'objectif de cette fonctionnalité était d'importer et exporter des formes au format PLY (Polygon File Format) principalement utilisé pour de la 3D, mais avec assez de flexibilité pour ajouter facilement une dimension et donc générer des figures en 4D.
Le programme lit des fichiers définissant les points et faces d'une figure, les interprète, et génère la forme correspondante. Il est aussi possible de sauvegarder l'état d'une forme au format PLY pour la visualiser plus tard.

En ce qui concerne l'environnement VR, l'objectif était de créer un espace où l'on pouvait interagir facilement avec les figures. C'est avec cette idée à l'esprit que nous avons ajouté un menu contextuel attaché à la main (hand menu: codé par Loup) où l'on peut sélectionner facilement les formes à afficher, et effectuer des opérations (rotation, homothétie) sur ces dernières.

Nous avons également factorisé du code, puisque l'ajout de la VR a généré des classes supplémentaires, et nous avons dû commencer à réfléchir à optimiser la construction des figures, avec notamment l'hypersphère qui était trop gourmande en ressource pour être affichée sur le casque (travail de Noah).

Itération 5 : Dernières fonctionnalités

Pour cette avant dernière itération, nous avons avancé et finalisé plusieurs tâches importantes du projet. La première a été le fait de faire fonctionner la projection 3D d'une forme 4D dans l'environnement VR sans problème (codé par Noah). Nous avons modélisé correctement la scène VR pour permettre à l'utilisateur de visualiser les différentes projections d'une forme 4D en un seul coup d'œil, en plaçant les projections les unes à côté des autres qui sont toutes visibles en même temps (codé par Noah).

Le seconde grosse nouveauté de cette itération est la mise en place du "Hand Menu" (codé par Loup). Ce menu a pour but de permettre à l'utilisateur d'interagir avec les formes 3D et 4D directement dans la VR, ce qui n'était pas possible avant. Le menu permet de changer la forme que l'on souhaite visualiser, de la modifier (rotation et homothétie) et aussi de faire apparaître ou non les arêtes et points de la forme ainsi que les faces de celle-ci.

Nous nous sommes également penchés sur l'optimisation de notre application, car l'affichage de certaines formes (principalement pour l'hypersphère) prenait beaucoup de temps à cause du nombre de points à afficher. Nous avons donc pris la décision de ne pas afficher les points et arêtes pour l'hypersphère et les voxels, tout en améliorant la partie calcul pour l'apparition des points (fait par Elias).

Le dernier ajout de cette itération sont les voxels en 4D (codé par Elias). Un voxel est une unité, l'équivalent 3D d'un pixel. C'est un cube comparé au pixel qui est un carré et est utilisé notablement dans Minecraft, un jeu où tout l'environnement est composé de voxels et où l'on peut construire n'importe quoi en 3D.

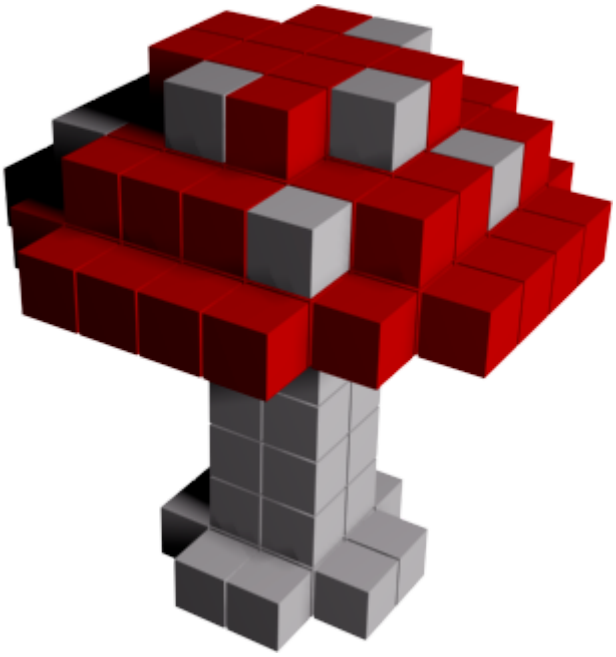

Figure n°9 - Exemple de figure composée de voxels (3D)

Dans notre projet, l'objectif était de générer des formes en 4D composées de voxels 4D, qui sont donc des hypercubes, afin que l'utilisateur puisse facilement créer des formes simples.

Pour se faire, il est possible d'appeler une méthode de chargement d'un fichier PLY donnant les positions de chaque voxel pour l'afficher dans l'espace en temps que figure qui peut être manipulée dans le programme. A titre d'exemple, voici l'équivalent 4D d'une croix générée à l'aide d'un fichier PLY ci-dessous :

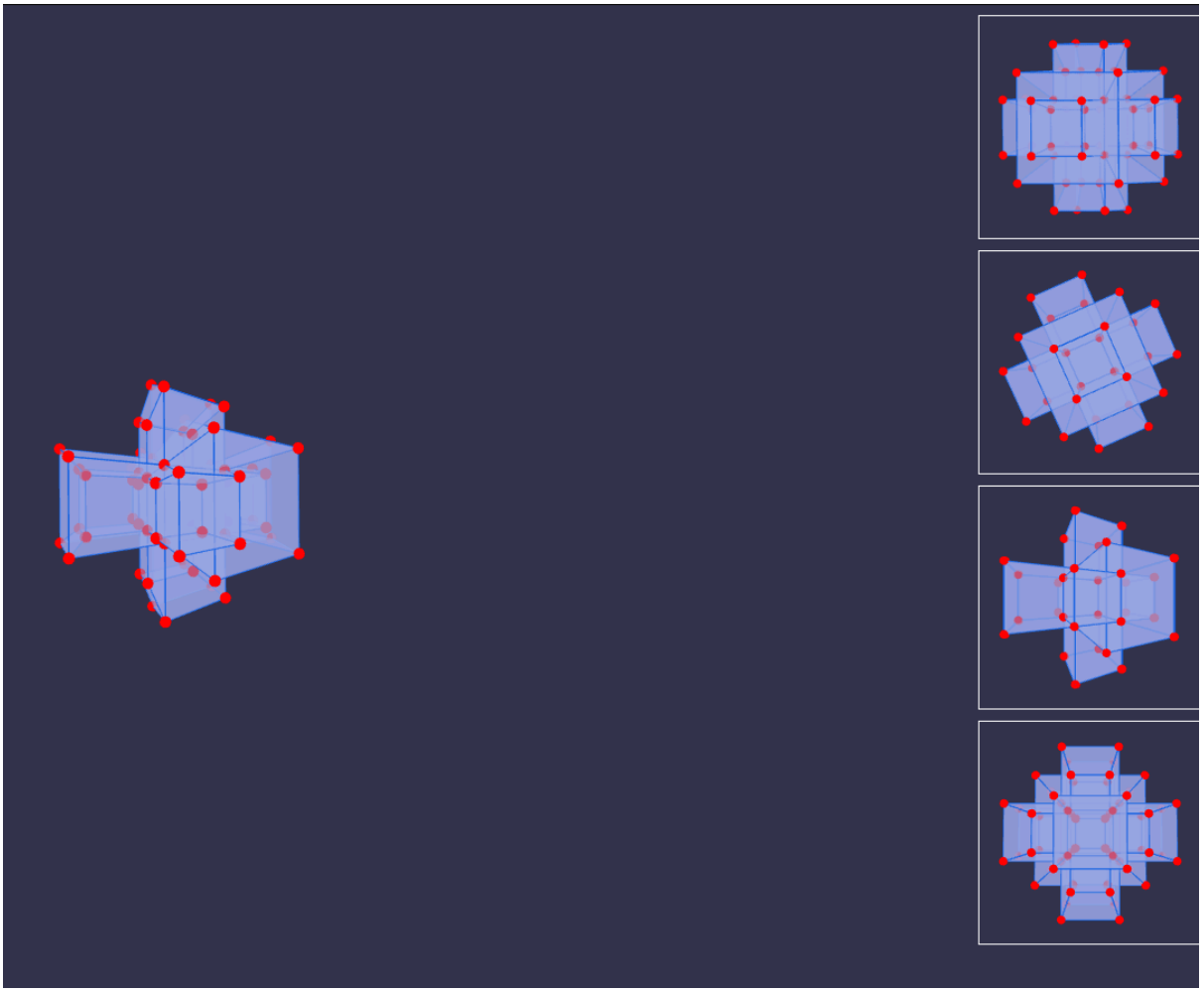

Figure 10 - Croix 4D affichée sur navigateur avec ses différentes projections 3D

```
ply
format ascii 1.0
comment Positions de voxels 4D pour former une croix 4D
element vertex 9
property float x
property float y
property float z
property float w
end_header
0 0 0 0
1 0 0 0
-1 0 0 0
0 1 0 0
0 -1 0 0
0 0 1 0
0 0 -1 0
0 0 0 1
0 0 0 -1
```

Figure 11 - Fichier PLY pour afficher une croix 4D en voxels

Itération 6 : Présentation aux BUT 1

Cette itération a été très courte en temps, car nous avions à préparer beaucoup de documentations et de rendus de fin de projet. Nous avons quand même pu ajouter quelques petites fonctionnalités à l'application, comme faire apparaître les faces de l'hypersphère (fait par Elias). Nous avons tenté d'ajouter une option pour déplacer le hand menu d'une main à l'autre, mais cet ajout est resté en phase de développement par manque de temps (fait par Loup).

Nous avons eu la possibilité de présenter notre projet tutoré aux étudiants de BUT 1. Nous sommes passés devant 3 groupes, avec entre 10 et 25 étudiants par groupe. Ces présentations nous ont permis de parler de notre sujet dans son intégralité, et de faire découvrir aux étudiants ce qui les attendait en 3ème année. Nous avons pu échanger avec certains qui étaient intéressés par la manière dont nous avons avancé dans ce projet exploratoire.

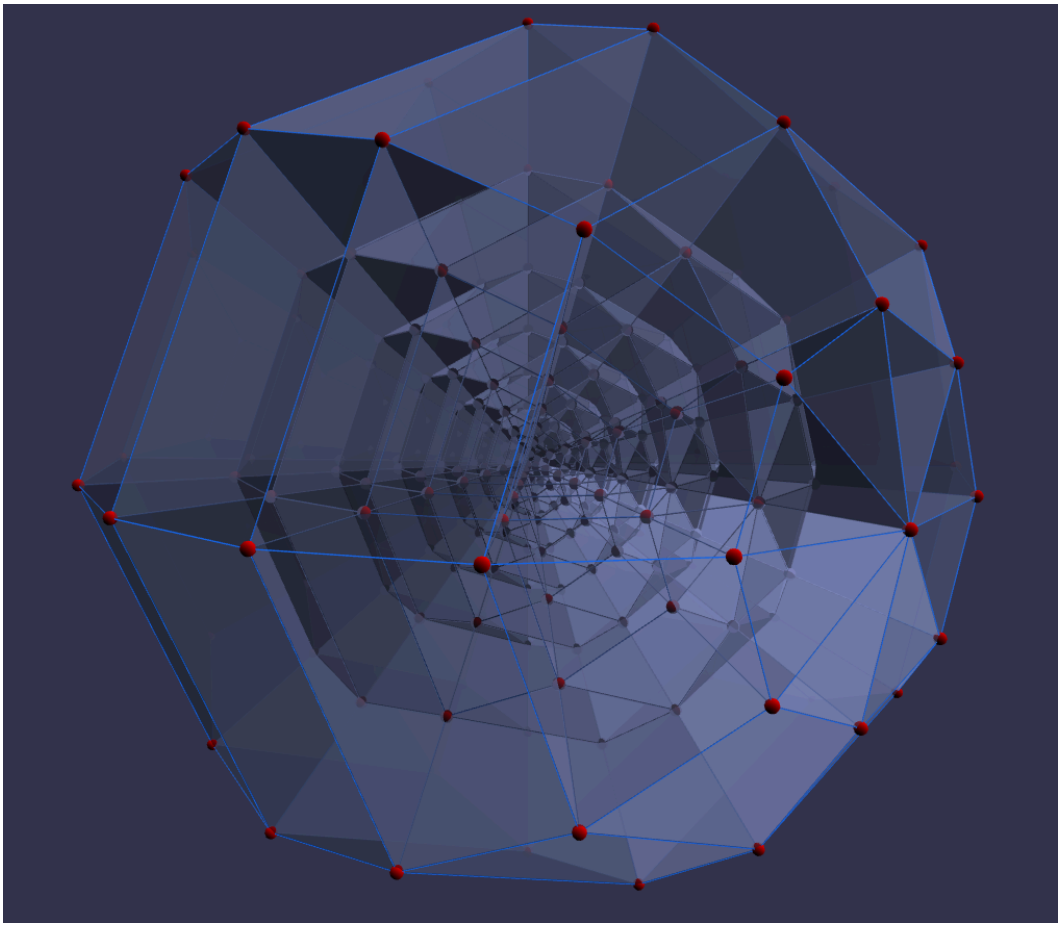

Figure n°12 - Affichage des faces de l'hypersphère

Avis du groupe sur le développement

ALLART Noah :

“Le passage de nos formes 4D à la VR est une fonctionnalité dont je suis particulièrement fier. En effet, j’ai beaucoup apprécié le fait d’utiliser un casque VR pour visualiser nos formes, ce qui m’a aussi motivé à effectuer des recherches plus approfondies sur le fonctionnement d’un casque VR. Combiner la VR avec de la 4D sur Babylon.js est un projet qui n’a jamais été fait ailleurs (de ce qu’on sait).”

ARMBRUSTER Loup :

“Une des fonctionnalités que j’ai le plus aimé voir accomplie est la rotation sur l’axe W de l’hypercube à la deuxième itération car nous avons réalisé du travail très théorique durant la première itération pour atteindre ce but. Voir que tout ce travail d’analyse mathématique et de découverte de la bibliothèque Babylon.js a été efficace et utile. Cela m’a vraiment donné un sentiment d'accomplissement et d'avancer dans ce projet. Cette fonctionnalité était aussi l’un des points critiques de notre application et pouvoir enfin observer les rotations de l’hypercube après en avoir autant parlé m’a permis de mieux comprendre la quatrième dimension.”

KHODJAOUI Elias :

“J’ai personnellement beaucoup aimé la façon dont nous implémentons BabylonJS. L’utilisation de classes javascript pour gérer la construction et la mise à jour des figures rend une grande partie du projet trivial, et rend la division des tâches relativement facile. En effet, même pour ajouter de nouvelles formes, il suffit de faire une nouvelle méthode pour générer les points, et notre code permet de gérer avec le système de classes toutes les opérations. Tant qu’aucune erreur n’est présente dans les méthodes de rotation, translation et mise à jour de la forme, la figure est directement fonctionnelle (comme avec le pentachore, qui fonctionnait très bien dès sa génération)”

Conclusion finale

Ce projet a été très intéressant et instructif, car nous avons dû explorer le sujet sans aucune connaissance au préalable (pour les parties 4D et VR). Nous avons choisi ce sujet, car la 4D était un principe que nous connaissions tous, mais de loin. Nous avons voulu en apprendre plus sur ce sujet que nous maîtrisons (partiellement) maintenant.

Monsieur Dupont nous a très bien expliqué les notions complexes en amont du développement, comme les projections, les matrices homogènes ou encore les problèmes de points à l'infini. Il a su être présent et à notre écoute tout au long des itérations.

Au global, le sujet a très bien avancé lors de sa conception grâce à la méthode agile imposée. Nous avons su avancer par étapes, progressivement, afin d'arriver à un produit fini et fonctionnel. Nous n'avons malheureusement pas pu implémenter toutes les fonctionnalités que nous aurions voulu (comme l'option du menu pour droitier ou gaucher), mais l'intégralité des attentes de monsieur Dupont et de nos tâches critiques ont été réalisées. Il est en effet intéressant de noter que toutes les fonctionnalités critiques et autres fonctionnalités mentionnées dans notre perspective de fin de projet après l'itération 3 ont été ajoutées. Nous avons choisi d'orienter nos efforts de développement lors des dernières semaines vers l'ergonomie de l'application en VR (optimisation des performances et menu interactif notamment).

Nous sommes très satisfaits de notre projet au vu de son ampleur et de son caractère exploratoire, qui nous a forcé à découvrir de nouvelles technologies dans le développement et nous forçant à optimiser notre code, puisque le casque VR Meta Quest 3 n'est pas très puissant : il n'a qu'une puce intégrée pour tous ses calculs, et elle n'est pas optimisée pour de la réalité augmentée. Mais malgré ces défis, notre projet tourne bien sur ce casque et est assez intuitif.

Nous pensons cependant que de par la nature exploratoire de ce projet, la fin de notre travail dessus ne signe pas la fin de vie de ce projet, qui peut très bien être repris par n'importe qui afin de l'améliorer.

Dans le cadre d'un autre projet tutoré, il pourrait être intéressant de se reposer sur les technologies implémentées pour pousser le projet plus loin. En effet, d'autres formes géométriques pourraient être ajoutées, une gestion de la 5D, 6D, …n-D pourrait être une autre piste de poursuite. Il serait également envisageable d'utiliser le code implémenté pour créer une vraie expérience en réalité virtuelle pouvant se présenter comme un jeu vidéo similaire à *Fez*, un jeu où l'on se déplace dans un environnement 2D, mais où l'on peut utiliser la 3D pour modifier l'environnement et progresser entre différents plateaux 2D, ou bien comme *4D Golf*, où modifier le circuit de golf peut révéler des chemins inattendus pour faire rentrer la balle dans le trou. On pourrait donc se déplacer dans un monde 3D, et passer par la 4ème dimension pour redécouvrir le même endroit mais sous un autre angle de vue.

Annexes

Documentation Babylon.js :
https://doc.babylonjs.com/

Exemples d'utilisation de Babylon.js :
https://playground.babylonjs.com/#WJXQP0
https://playground.babylonjs.com/#TJIGQ1#3
https://playground.babylonjs.com/#ja1nd3#942

Lien de la vidéo YouTube de l'application utilisant Unity :
https://youtu.be/1Zamk9ZOrjk?si=dRbxmKKnmsOTaYjZ

Définition d'un voxel :
https://fr.wikipedia.org/wiki/Voxel

Lien du jeu Fez :
https://store.steampowered.com/app/224760/FEZ/

Lien du jeu 4D Golf :
https://store.steampowered.com/app/2147950/4D_Golf/

Aides pour les potentiels futurs étudiants / développeurs :

- Documentation intégrée au code
- Mode d'emploi (fichier PDF) contenant toutes les informations relativent à la mise en place du serveur HTML en HTTPS